

CS3002E Compiler Design

Monsoon 2025

Lecture #11

Top-Down Parsing- 4

Department of Computer Science & Engineering
NIT Calicut

LL(1) Grammar

A grammar for which a predictive parsing table can be built with one symbol look-ahead, is said to belong to a class of grammars known as $LL(1)$ Grammars.

- first L stands for scanning input from left to right
- second L for doing a leftmost derivation
- 1 for one symbol look-ahead (in general, $LL(k)$ with k symbol look-ahead)

Grammar that is not LL(1): Example

$$E \rightarrow id + id \mid id * id$$

- FIRST sets of both the production bodies contain id
- attempt to construct a predictive parsing table results in two productions in table entry $T[E, id]$
- grammar is not suited for predictive parsing

In general, if there are productions, $A \rightarrow \alpha \mid \beta$, such that $\text{FIRST}(\alpha)$ and $\text{FIRST}(\beta)$ are not disjoint, then the grammar is not LL(1).

Check if LL(1)

$$\begin{aligned} S &\rightarrow iEtSeS \mid iEtS \mid a \\ E &\rightarrow b \end{aligned}$$

Grammar that is not LL(1): Example

$$S \rightarrow iEtSeS \mid iEtS \mid a$$

$$E \rightarrow b$$

Does Left-factoring result in an equivalent LL(1) grammar?

Grammar that is not LL(1): Example

$$S \rightarrow iEtSS' \mid a$$

$$S' \rightarrow eS \mid \epsilon$$

$$E \rightarrow b$$

Two productions in $T[S', e]$

$$S' \rightarrow eS$$

$$S' \rightarrow \epsilon$$

LL(1) Grammar: conditions

For productions $A \rightarrow \alpha \mid \beta$,

- $FIRST(\alpha)$, $FIRST(\beta)$ should be disjoint.
- If ϵ is in $FIRST(\alpha)$, then $FIRST(\beta)$ should not contain any terminal in $FOLLOW(A)$
- If ϵ is in $FIRST(\beta)$, then $FIRST(\alpha)$ should not contain any terminal in $FOLLOW(A)$

Left-recursive Grammar

$$A \rightarrow Aa \mid b$$

Multiple entries in the parsing table. Not LL(1).

Can be converted to an equivalent right recursive grammar.

$$A \rightarrow bA'$$

$$A' \rightarrow aA' \mid \epsilon$$

Eliminating Left recursion

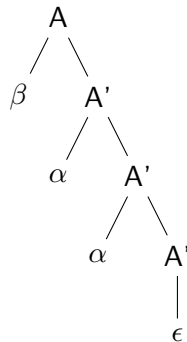
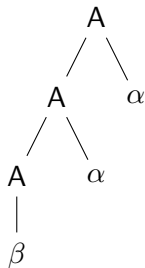
$$A \rightarrow A\alpha \mid \beta$$

an equivalent grammar (right recursive)

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \epsilon$$

Parse Trees for $\beta\alpha\alpha$



Test if LL(1)

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow id \end{aligned}$$

Eliminate Left Recursion

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow id \end{aligned}$$

Eliminating Left Recursion

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow id \end{aligned}$$

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid \epsilon \\ T &\rightarrow id \end{aligned}$$

Exercise

Convert the following to an equivalent LL(1) grammar and show the steps of a non recursive predictive parser for the input $id * (id + id)$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

Error recovery (optional)

Read ALSU section 4.4.5: Error recovery in predictive parsing

Reference:

- Aho A.V., Lam M.S., Sethi R., and Ullman J.D. Compilers: Principles, Techniques, and Tools (ALSU). Pearson Education, 2007.

Further reading:

- ALSU chapter 4, sections 4.3.3, 4.3.4, 4.4.3, 4.4.5 (optional)